

Predictive Maintenance Project Final Report

Date: 2026-07-04

Submitted by: Nikhilesh Mehta

Executive Summary

This final report documents the development of a predictive maintenance solution for vehicle engine failure prediction. The work completed establishes the business framing, registers the engine dataset to the Hugging Face dataset hub, prepares train and test data for supervised learning, and implements an end-to-end MLOps workflow including data registration, experiment tracking, model deployment, and CI/CD automation via github.

1. Introduction

A. Business Context and Objective

Unexpected engine failures generate repair costs, operational downtime, and safety risk for both individual owners and fleet operators. The business objective of this project is to use engine sensor data to classify whether an engine is operating normally or requires maintenance intervention. A successful model would support more proactive service scheduling, reduce unplanned breakdowns, and improve the reliability of engine-powered assets.

The interim scope focuses on building the project foundation needed for predictive maintenance: establishing a governed data source, preparing analysis-ready data, and defining a reproducible experimentation framework for model comparison.

B. Limitations

Since the dataset does not contain time-series information, proactive failure prediction is not feasible. Therefore, the scope of this project is limited to reactive prediction based on current sensor readings.

2. EDA insights

The source file currently contains 19,535 records across seven columns, including six predictive input variables and one binary target. There are no missing values or duplicate records in the dataset. The class distribution is moderately imbalanced, with class 0 representing 36.95% of records and class 1 representing 63.05% of records. Because of this imbalance, a stratified split is appropriate to preserve class proportions in both training and testing partitions. The resulting split sizes are 15,628 records for training and 3,907 records for testing.

Table 1. Engine Sensor Variables Used in the Predictive Maintenance Dataset

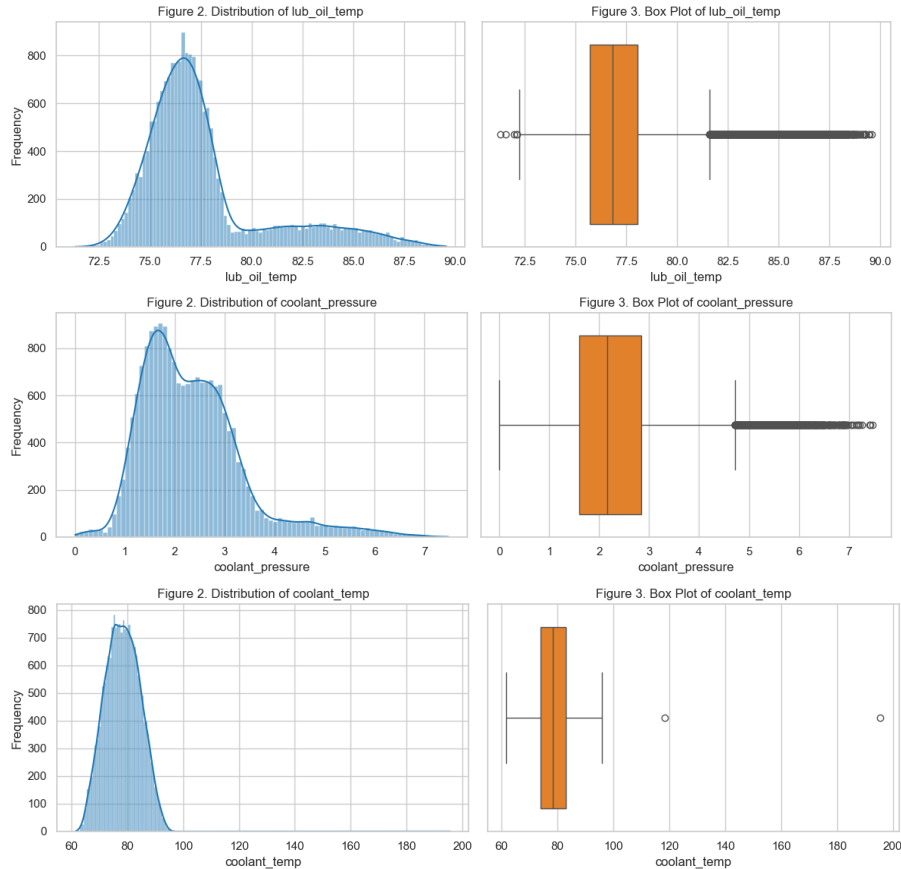
Variable	Type	Description
Engine rpm	Continuous	Revolutions per minute of the engine; proxy for engine operating intensity.
Lub oil pressure	Continuous	Lubrication system pressure used to assess internal engine protection.
Fuel pressure	Continuous	Fuel delivery pressure associated with combustion stability.
Coolant pressure	Continuous	Cooling system pressure related to thermal regulation.
lub oil temp	Continuous	Lubricating oil temperature used to monitor heat and viscosity conditions.

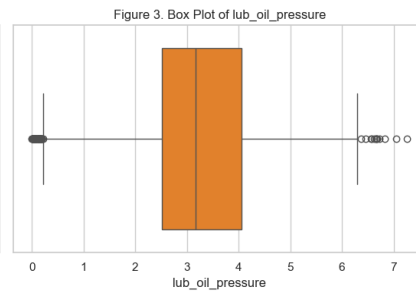
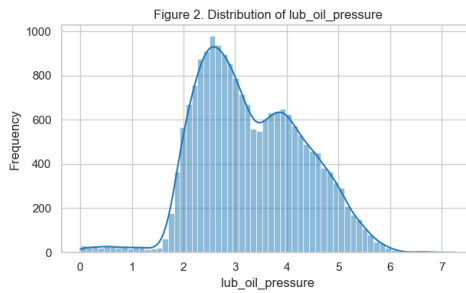
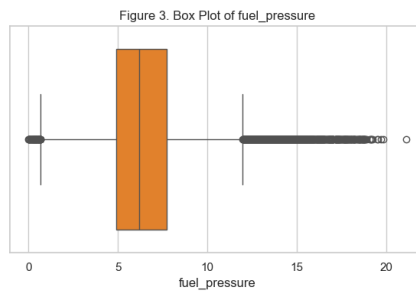
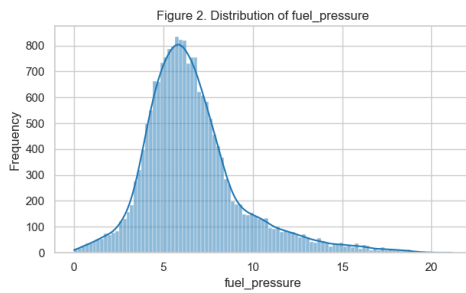
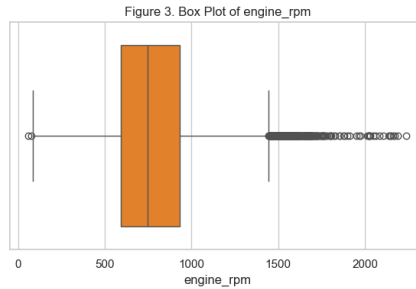
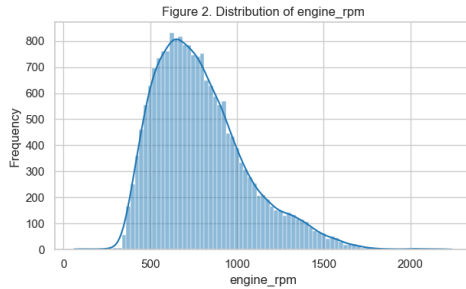
Coolant temp	Continuous	Coolant temperature used to detect thermal stress and overheating risk.
Engine Condition	Binary target	Target variable where 0 and 1 represent alternate operating states used for predictive maintenance classification.

A. Univariate Analysis

- engine rpm, fuel pressure, and coolant_pressure are right-skewed with visible high-end outliers.
- lub_oil_temp is concentrated in a narrow range around the mid-to-high 70s with a right tail.
- coolant_temp is concentrated near 70-85, with rare extreme outliers (including one very high value).

Figure 1. Univariate Analysis

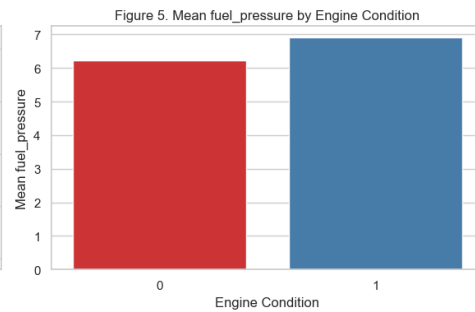
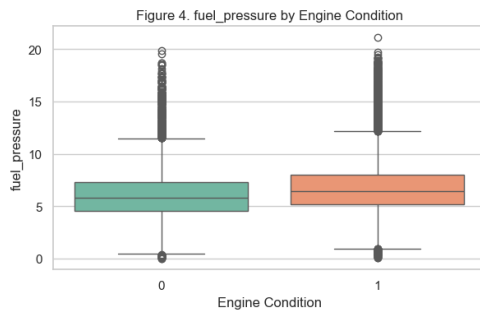
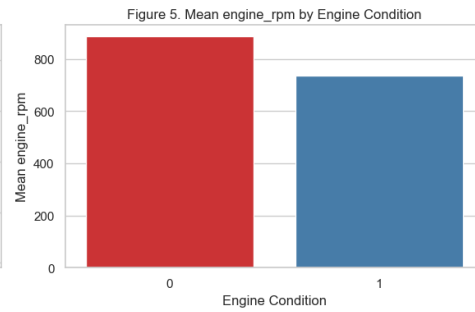
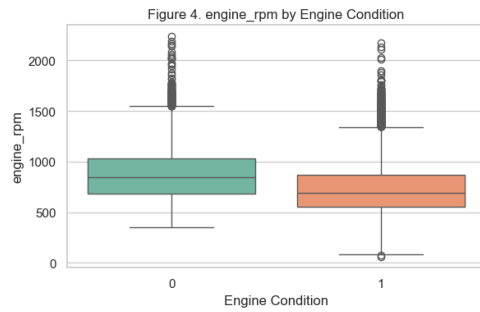
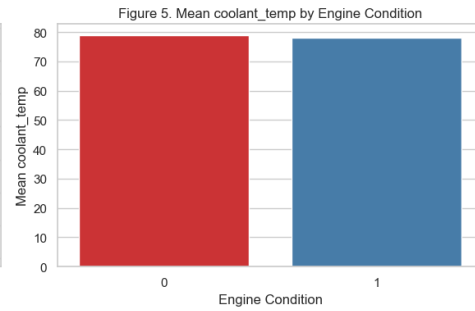
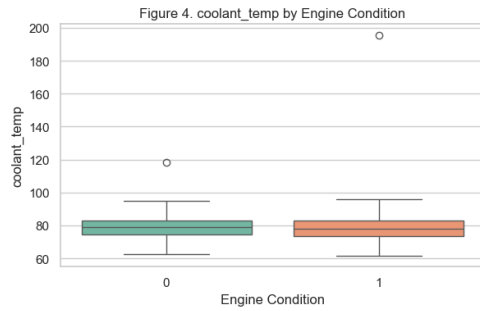
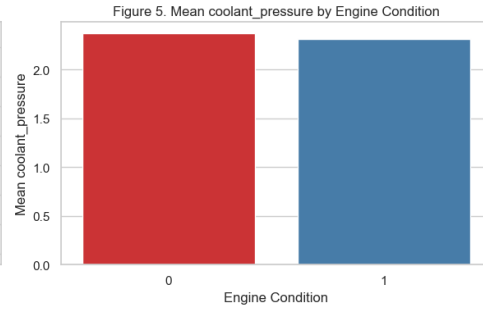
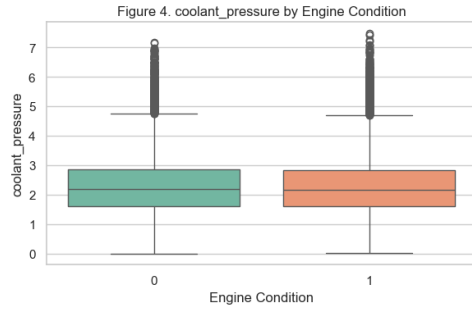


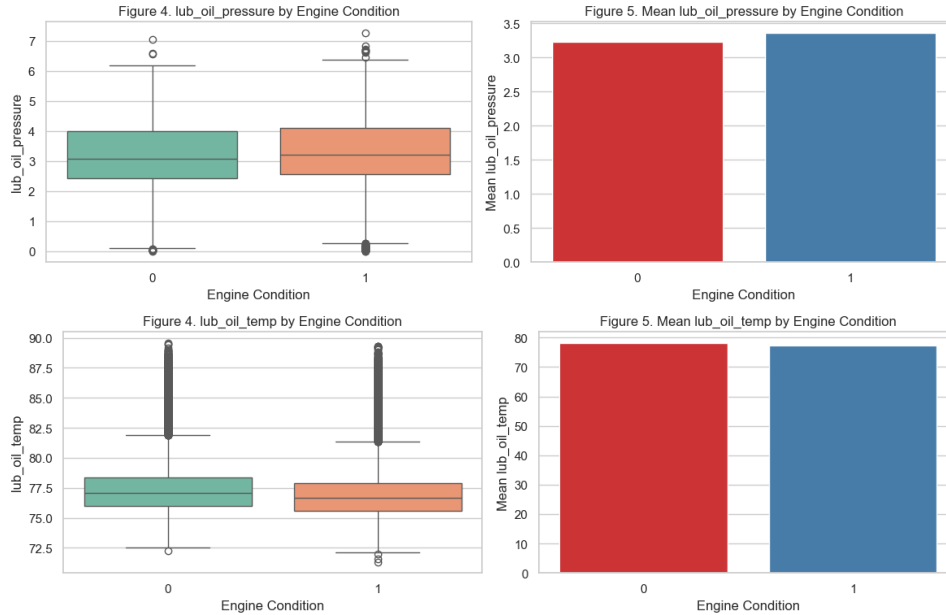


B. Bivariate Analysis (vs Target: engine_condition)

- engine_rpm: class 1 tends to have lower RPM than class 0.
- fuel_pressure tends to be higher for class 1 than class 0.
- lub_oil_pressure shows a mild upward shift for class 1.
- Temperature variables (lub_oil_temp and coolant_temp) have weaker class separation in mean-level comparison.

Figure 2. Bivariate Analysis

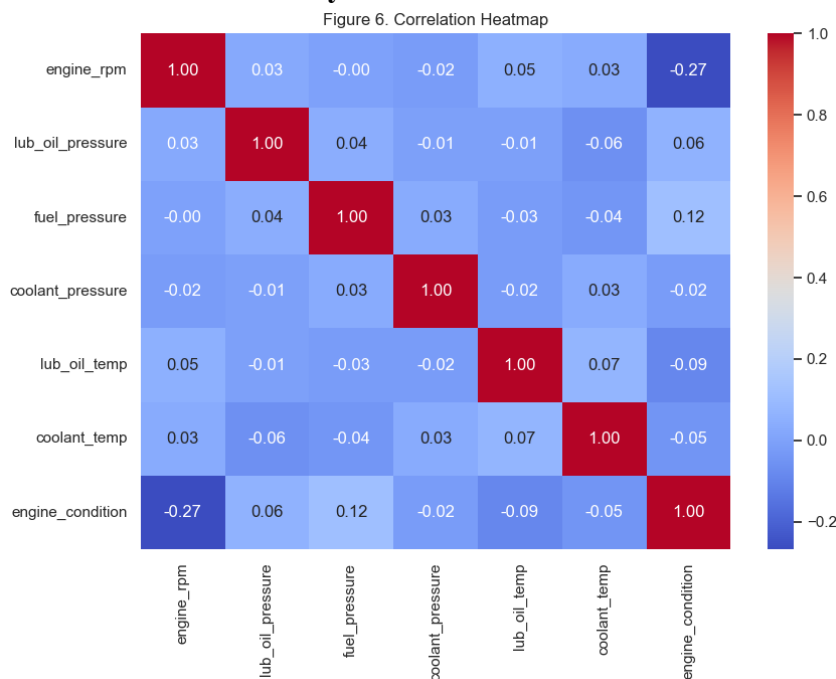




C. Multivariate Analysis

- Correlation heatmap indicates generally weak-to-moderate linear relationships among predictors.
- Strongest association with target is negative for engine_rpm (about -0.27) and positive for fuel_pressure (about +0.12).
- Pairwise plots suggest non-linear boundaries, supporting tree-based models (As in the notebook file)
-

Figure 3. Multivariate Analysis



Insights/Observations based on EDA

- Data quality is strong (no missing/duplicate issues), so minimal cleaning is required.
- Class imbalance exists and should be considered in evaluation (F1/Recall more informative than accuracy alone).
- engine_rpm and fuel_pressure appear to be the most informative early predictors.
- Outliers are present in multiple sensors; Tree-based ensemble methods are expected to perform well due to their robustness to outliers and non-linear feature interactions.
- The EDA findings support the selection of a Decision Tree model as the baseline classifier for subsequent experimentation.

3. Model Selection and Deployment Methodology

We have implemented an end-to-end model experimentation process using a Decision Tree classifier tuned with GridSearchCV. The model was optimized over a broad hyperparameters using 5-fold cross-validation with F1-score as the primary objective to account for class imbalance. All key run details, including tuned parameters, cross-validation results, holdout test metrics, and model artifacts, were tracked in MLflow (local tracking file due to network restrictions). The best-performing model was then serialized and registered to the centralized Model Hub repository, ensuring traceability, reproducibility, and deployment readiness for predictive maintenance operations.

A. Model selection Steps

Table 2. Dataset Profile and Train-Test Split Summary

Item	Value
Total observations	19,535
Total columns	7
Predictor columns	6
Target column	Engine Condition
Class 0 count	7,218 (36.95%)
Class 1 count	12,317 (63.05%)
Training set size	15,628 rows
Testing set size	3,907 rows
Split strategy	Stratified 80/20 split with random_state=42

A Decision Tree model was selected because:

1. It performs well on tabular operational data
2. It is robust to non-linear feature relationships
3. It is highly interpretable for business stakeholders
4. It supports practical maintenance-rule extraction

Hyperparameter optimization was conducted using GridSearchCV with 5-fold cross-validation and F1-score as the primary optimization metric (important for class-imbalanced scenarios).

Hyperparameter	Values Tested	Rationale
criterion	gini, entropy, log_loss	Compare split quality measures
max_depth	3, 5, 7, 10, None	Control overfitting vs learning complexity
min_samples_split	2, 5, 10, 20	Avoid fragile splits on small sample sizes
min_samples_leaf	1, 2, 4, 8	Improve generalization by enforcing minimum leaf support
class_weight	None, balanced	Address class imbalance effects
ccp_alpha	0.0, 0.001, 0.01	Cost-complexity pruning for simpler trees

Table 3. Planned Evaluation Metrics for Experiment Tracking

Metric	Purpose in This Project
Accuracy	Overall share of correct predictions across maintenance classes.
Precision	Reliability of predicted maintenance flags; useful when false alarms are costly.
Recall	Ability to identify engines that require maintenance; useful when missed failures are costly.
F1-score	Primary tuning metric balancing precision and recall for imbalanced binary classification.
ROC-AUC	Ranking ability across classification thresholds for comparative model assessment.

B. Deployment Steps

Cloud data integration

The preparation workflow reads the source dataset from the Hugging Face dataset space, identifies the binary target as `engine_condition`, and includes validation checks for missing values and standardizes the dataset schema. Numeric columns are imputed with the median, while any categorical columns are imputed with the mode and encoded only if they exist. The workflow then creates a 80/20 train-test split using a fixed random seed and saves both split files locally before uploading them back to the Hugging Face dataset repository.

Experiment Tracking

MLflow is used for experiment tracking during this stage of the project. A local MLflow UI server is configured at <http://127.0.0.1:5000>. The following were logged for each best run:

1. Model name
2. Best hyperparameters
3. Best cross-validation F1 score
4. Test metrics (Accuracy, Precision, Recall, F1, ROC-AUC)
5. Serialized model artifact

Table 4. MLflow experiment summary for the tuned Decision Tree model

Experiment	Run name	Run ID	Accuracy	Best CV F1	Test F1	Precision	Recall	ROC-AUC	Key params
Predictive_Maintenance_Vehicle	DecisionTree	8e087de0a1934ae2806e792abd7d69da	0.6575	0.7640	0.7613	0.6790	0.8664	0.6721	ccp_alpha=0.001, criterion=gini, max_depth=5, min_samples_leaf=1, min_samples_split=2, class_weight=None

This provides full traceability from training setup to final model artifacts.

Model Registration and Deployment Readiness

After the model tuning and validation phases were completed, the best-performing model was prepared for deployment by following a structured model registration process:

- 1. Model Serialization and Artifact Creation**

The final selected model, along with its learned parameters and associated configuration, was serialized into a portable artifact. This ensured that the trained model could be consistently reloaded for inference without requiring retraining, while preserving reproducibility across different environments.

- 2. Model Registration in Hugging Face Model Hub**

The serialized model artifact was uploaded to the Hugging Face Model Hub, where it was registered as a versioned repository. Storing the model in the Model Hub provides centralized version control, secure access, and standardized APIs for model retrieval. It also enables seamless integration with downstream applications, CI/CD pipelines, and collaborative workflows.

This registration process established a reusable and version-controlled model registry endpoint that can be leveraged for production deployment, batch inference, or integration into machine learning pipelines. Additionally, maintaining the model in a centralized repository simplifies future updates, rollback to previous versions, and governance of the model lifecycle.

Model Deployment (On Hugging Face Space)

The trained Decision Tree model was deployed as an interactive web application using **Streamlit** on **Hugging Face Spaces**, providing an intuitive interface for real-time engine health prediction. This deployment enables end users to submit engine telemetry data through a browser and receive immediate inference results without requiring access to the underlying model implementation.

Deployment Components

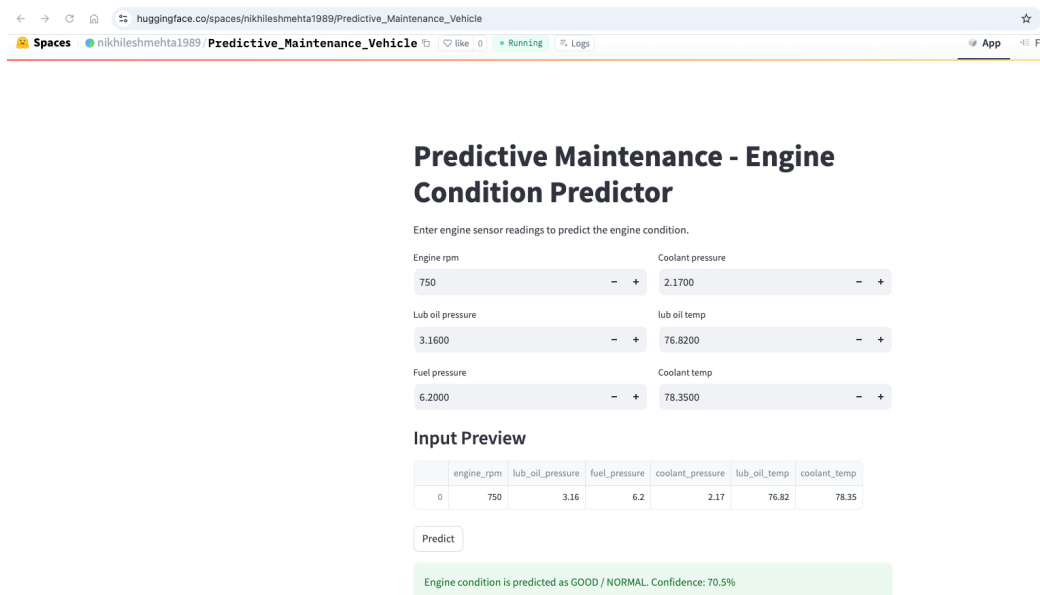
- Application Entry Point:**

The deployment is implemented through the `deployment/app.py` script, which serves as the primary entry point for the Streamlit application. The application initializes the user interface, processes user inputs, loads the trained model, and generates prediction results.

- Model Loading and Artifact Management:**
 The trained model artifact is dynamically retrieved from the Hugging Face Model Hub using the `hf_hub_download` API with token-based authentication. This approach ensures that private model assets remain secure while allowing the deployment to always access the latest versioned model artifact without embedding the model directly within the application.
- Input Processing and Feature Preparation:**
 The application applies the same normalization and feature transformation steps used during model training, ensuring that the input data conforms to the expected feature schema. This guarantees consistency between the training and inference pipelines and helps maintain prediction accuracy.
- Real-Time Prediction Workflow:**
 Once preprocessing is complete, the transformed feature vector is passed to the deployed Decision Tree model to generate an engine health prediction. The predicted maintenance status is then displayed to the user through the Streamlit interface, enabling fast and interactive decision support.

Overall, this deployment architecture provides a **reproducible, scalable, and user-friendly inference layer** that directly consumes versioned model artifacts from the Hugging Face Model Hub. By separating model storage from application logic and maintaining a consistent preprocessing pipeline, the solution supports reliable deployment, simplified model updates, and seamless integration into future production or batch inference workflows.

Figure 4. Hugging face application (Streamlit Application)



GitHub Actions Workflow Automation

To automate the end-to-end machine learning lifecycle, a Continuous Integration and Continuous Deployment (CI/CD) workflow was implemented using GitHub Actions. The workflow configuration is defined in `.github/workflows/pipeline.yaml` and is automatically triggered whenever a commit is pushed to the **main** branch.

This automation ensures that every approved code or configuration change executes a standardized MLOps pipeline, eliminating the need for manual intervention and maintaining consistency across all pipeline executions.

The workflow orchestrates the complete sequence of tasks required to prepare data, train and validate the model, register artifacts, and update the deployment environment. By integrating these stages into a single automated pipeline, the solution promotes reproducibility, traceability, and reliable model delivery.

Stage 1: Data Registry

The workflow begins by validating the source dataset and uploading it to the Hugging Face Dataset Hub. Registering the dataset in a centralized repository provides version control, making it possible to associate every model with the exact dataset version used during training. This stage establishes a reliable data registry that supports collaboration, reproducibility, and future model retraining.

Stage 2: Data Preparation

Once the dataset is registered, the workflow executes the data preprocessing pipeline. This stage performs missing value imputation, validates the dataset schema to ensure feature consistency, and applies any required preprocessing transformations. Finally, the dataset is partitioned into 80% training and 20% testing subsets, ensuring a consistent and repeatable data preparation process for each pipeline execution.

Stage 3: Model Building

During the model-building stage, the pipeline trains the machine learning model using the prepared training dataset. Hyperparameter tuning is performed to identify the optimal model configuration, and the trained models are evaluated using the testing dataset. Throughout this process, MLflow records experiment metadata, training parameters, evaluation metrics, and serialized model artifacts. This experiment tracking capability enables easy comparison between different model versions, simplifies reproducibility, and provides a complete audit trail of the model development process.

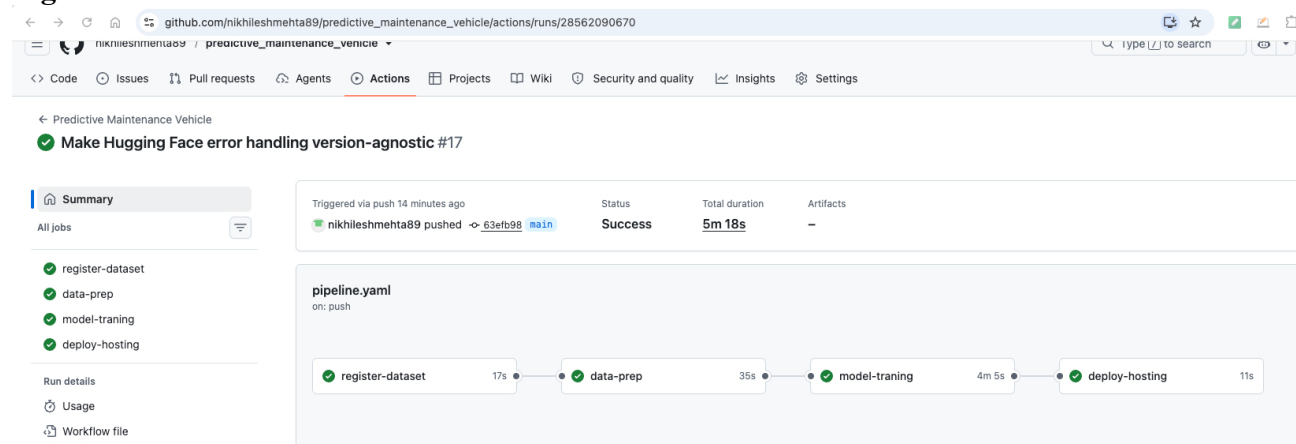
Stage 4: Deployment Host Update

After the best-performing model has been selected and validated, the workflow packages the serialized model artifact and updates the deployment environment. The latest model artifact is uploaded to the Hugging Face Model Hub, allowing the deployed Streamlit application hosted on Hugging Face Spaces to retrieve the newest validated model for inference. This automated deployment process ensures that the production application remains synchronized with the latest approved model version while minimizing deployment errors and downtime.

Benefits of Workflow Automation

The GitHub Actions workflow establishes a fully automated, reproducible, and scalable MLOps pipeline that integrates data versioning, preprocessing, model training, experiment tracking, model registration, and deployment into a single continuous workflow. Automating these tasks significantly reduces manual effort, minimizes the risk of configuration inconsistencies, accelerates model release cycles, and ensures that every approved code change results in a production-ready machine learning solution. Furthermore, the integration of GitHub Actions with Hugging Face Hub and MLflow provides comprehensive version control, experiment traceability, and seamless deployment, supporting efficient collaboration and long-term maintainability of the MLOps system.

Figure 5. Github Action Workflow



Business-Ready Actionable Insights

1. Failure risk is detectable early from sensor patterns. Use the model score as an early-warning signal to identify engines likely to require maintenance before a breakdown occurs.
2. Prioritize maintenance by risk tier instead of fixed schedules. Create three operational tiers:
 - High risk: inspect within 24 hours
 - Medium risk: inspect in 3-7 days
 - Low risk: continue routine monitoring

3. Lubrication and pressure variables are critical levers. Patterns in Lub oil pressure, Fuel pressure, and Coolant pressure are strong contributors to classification outcomes, so these should be monitored with tighter operational limits.
4. Data quality directly impacts prediction reliability. Standardize sensor calibration and logging frequency across all equipment to reduce noisy readings and improve model consistency.

Recommendations for Implementation

1. Deploy a live risk dashboard for maintenance planners. Show per-engine risk class, confidence score, last reading time, and recommended action window.
2. Integrate model alerts into maintenance workflow. Automatically create a maintenance ticket when predicted risk exceeds the chosen threshold.
3. Tune decision thresholds based on business cost. Set thresholds to minimize total cost, balancing:
 - False negatives (missed failures, high downtime cost)
 - False positives (unnecessary inspections, labor cost)
4. Start with a pilot and scale in phases. Run a 6-8 week pilot on a subset of engines, compare outcomes with historical maintenance, and then expand plant-wide.
5. Retrain on a fixed cadence and monitor drift. Retrain monthly or quarterly, and trigger early retraining if feature distributions shift materially.

These recommendations convert model output into operational decisions, measurable savings, and a repeatable maintenance strategy.

5. Risks and mitigations

Risk	Potential Impact	Mitigation Implemented	Residual Risk
Overfitting from deep trees	Poor real-world performance	max_depth, min_samples_leaf, ccp_alpha tuning	Medium
Imbalanced class bias	Missed failure cases	F1 optimization and class_weight search	Low to medium
Metric over-reliance	Misleading model acceptance	Multi-metric evaluation on holdout test	Low
Tracking gaps	Poor auditability	MLflow parameter/metric/artifact logging	Low
Model version confusion	Deployment inconsistency	Model Hub registration and centralized storage	Low

Literature Review:

[1] Andreas Theissler, Judith Pérez-Velázquez, Marcel Kettelgerdes, Gordon Elger - Predictive maintenance enabled by machine learning: Use cases and challenges in the automotive industry, Reliability Engineering & System Safety, Volume 215, 2021

[2] Shib Shankar Golder, Sujan Das , Somnath Mondal - Predictive Maintenance in Automotive Manufacturing: Leveraging Big Data, Advanced Algorithms, and AI for Early Failure Detection Through Engine Heartbeat Analysis, Indian Library Science, Vol. 44 No. 6 (2024): LIB PRO. 44(6), JUL-DEC 2024 (20-12-2024)